Performance of the kernel and the Prelude

This is the working plan for performance work. It is deliberately measurement first; the estimates below are ranges, not promises.

Where the time goes

The kernel is the native execution artifact: `components/kernel/lowered-abstraction.fs`, `components/kernel/store.fs`, `components/kernel/tensor.fs`, `components/kernel/types.fs`, `components/kernel/kernel-state.fs`, and the ABI. The Prelude is the F# reference and the current product interpreter: `components/prelude/evaluator/evaluator.fs`, `common.fs`, `dispatch.fs`, `closure.fs`, and `components/prelude/scheduler.fs`. `--prose` runs the kernel `LoweredAbstraction` VM with `EvaluatorKernelBridge` callbacks into the Prelude, so both sides are on the hot path.

Hot spot	Where	Why it is expensive	Fix / effort
The store	`kernel/types.fs`, `store.fs`	`Lexicon` is an immutable `Map<EntryIdentity, Tensor>`. Every literal and arithmetic result inserts a map node; every lookup walks a tree. `Store` is a record copied on each update.	Dense arena or `Dictionary`/array by `NextIndex`; structural sharing only at snapshots. **3-5 weeks**.
The operand stack	`kernel/stack.fs`, `lowered-abstraction.fs`	A linked list with cons cells; `Stack.depth` is linear and is called on failures. Slots are a `Map<Slot, EntryIdentity>` updated on every store/load.	Array/`ResizeArray` stack and an `EntryIdentity[]` slot frame; a program counter integer. **1-2 weeks**.
Instruction dispatch	`lowered-abstraction.fs`	A DU `match` per instruction, `Result` allocations, and a `Map` for slots. The VM is a recursive interpreter, not a flat loop.	Flat instruction array, integer opcode switch, struct status, no per-instruction `Result`. **1-2 weeks**.
The host boundary	`prelude/evaluator/`	Each primitive converts `KernelState` to `EvaluationState` and back, copying a large record twice.	A `KernelPrimitive` interface

Hot spot	Where	Why it is expensive	Fix / effort
	`kernel-bridge.fs`		that operates on `KernelState` directly, with the Prelude state behind one pointer. **1 week**.
The evaluator loop	`prelude/evaluator/evaluator.fs`	`Program: TermIr.Term list`, `@` splices, a large `EvaluationState` record copied per token, and `Location`/`Position` updates.	Array/`ResizeArray` program, smaller hot state, prepend/reverse instead of append. **2-4 weeks with the store work**.
Arithmetic	`prelude/evaluator/common.fs`	`Array.map2`, intermediate lists, and a fresh result tensor per operation.	`Span<double>`/ one result array; elementwise kernels. **1 week**.
Output and text	`evaluator.fs`, `store.fs`	`Output @ [ line ]` is O(n); text is stored as `double[]` chars and cached in `Map`/`Set`.	Append via a reversed list or `ResizeArray`; text handles in a dictionary. **3-5 days**.
Dispatch	`prelude/evaluator/dispatch.fs`, `scheduler.fs`	135 explicit word cases, generic/method maps, and a scheduler state with several maps.	Table-driven dispatch and cached generic targets; arrays/ dictionaries

Hot spot	Where	Why it is expensive	Fix / effort
			for actors. **1-2 weeks**.

Staged plan

1. **Stage 0 — measure.** The JIT harness is the `--benchmark` mode of the test deployable (`projects/scale-tests`). It pins kernel arithmetic, interpreter arithmetic, a prose book cast, and text interning; it records wall time and `GC.GetAllocatedBytesForCurrentThread`. The committed snapshot is `docs/performance-baseline.typ`. The AOT smoke is later. **2-3 days.**
2. **Stage 1 — low-risk wins.** Stack/slots/program arrays, output append, `Span` arithmetic, no `Array.toList` in `findPair`, and the bridge fast path. Expected 1.5–3x on arithmetic and stack workloads. **1-2 weeks.**
3. **Stage 2 — the value/store model.** Decide between (a) a dense arena indexed by `EntryIdentity` with snapshots, or (b) an unboxed `Value` stack with only text/tensors in the store. (b) is the larger semantic change; (a) fits the append-only image and the M1 arena direction better. Expected a further 1.5–3x. **3-5 weeks, with persistence/history/ownership tests.**
4. **Stage 3 — the VM and the writer.** Flat opcode array, integer program counter, then the P2 SRM writer for hot units. The writer is the path to 10x or more; it is JIT-only and excluded from AOT (D84, D120). **3-5 weeks.**
5. **Stage 4 — scheduler and dispatch.** Arrays/dictionaries for actors and channels, cached generic/method targets. **1-2 weeks.**

Total. A focused 8–12 week program for an interpreter that is roughly 2–4x faster and a compiled path that is much faster than that. The first measurable improvement should land in 1–2 weeks. These estimates assume one experienced F# developer with the existing suite as the regression gate and no scope expansion.

Rules

- Never optimize without a benchmark that fails the old code and passes the new one. A performance claim without a measured workload is a story.
- Keep the kernel free of Prelude types. A fast kernel that grows a dependency on `EvaluationState` has lost the architecture.
- Prefer data-oriented changes that preserve the image invariant. A mutable arena is acceptable if persistence still serializes the image data and a reader re-threads the roots.
- Re-run the full suite and the differential corpus after any representation change. The compiler is the authority; the benchmark is the evidence.